

YACHAY TECH UNIVERSITY

School of Mathematical and Computational Sciences

Distributed Systems

Semester II 2026

Workshop 1: Client/Server

Group Members:

Alejandro Ortiz

Demian Viteri

Professor:

Prof. Francisco Hidrobo

August 20, 2026

1 Objectives

- Understand and implement client-server communication using sockets.
- Demonstrate successful communication across different hosts
- Implement multithreading to handle concurrent clients, automate random message generation, and prove interoperability between programs written in different programming languages (Python and C).

2 Environment and Configuration

- **Operating Systems:** Windows 11 (Host 1) and Linux (Host 2).
- **Programming Languages:** Python 3.x and C (compiled with GCC/Cygwin).
- **Software Tools:** Visual Studio Code, Git, Github.
- **Network Configuration:** Both hosts were connected to the same Local Area Network (LAN). The server was configured to listen on port 12000. Windows Firewall rules were modified to allow inbound TCP traffic on the specified port.

3 Implementation

In both Python and C, the server was modified to implement a multithreaded architecture. Instead of blocking the main execution loop, a new thread is spawned using `threading.Thread` in Python and `pthread_create` in C for each incoming connection.

For the client-side automation, we generated a random integer between 1 and 3 to determine the number of messages. Each message payload was constructed using a randomly generated 8-character string.

Listing 1: Client socket implementation in Python

```
1 def main() -> None:
2     numberOfClients:str = input("Enter the number of clients: ")
3     if not numberOfClients:
4         numberOfClients_int = DEFAULT_NUMBER_OF_CLIENTS
5         logging.info(f"No number of clients specified. Using {
6             DEFAULT_NUMBER_OF_CLIENTS} clients.")
7     else:
8         numberOfClients_int = int(numberOfClients)
9         serverName, serverPort = serverIdentification()
10        threads:list = []
11        for _id in range(numberOfClients_int):
12            thread:threading.Thread = threading.Thread(target =
13                clientInitialization, args = (_id + 1, serverName, serverPort
14                ))
15            threads.append(thread)
16            thread.start()
17        for thread in threads:
18            thread.join()
19        logging.info("All clients have finished.")
```

Listing 2: Server socket implementation in Python

```

1  def handle_client(connectionSocket, addr):
2      print(f"From Client: {addr}")
3      try:
4          sentence = connectionSocket.recv(1024).decode()
5          print(f"I received from {addr}: {sentence}")
6          capitalizedSentence = sentence.upper()
7          time.sleep(3)
8          connectionSocket.send(capitalizedSentence.encode())
9      except Exception as e:
10         print("Error with client", addr, ":", e)
11     finally:
12         connectionSocket.close()
13
14 try:
15     serverPort = int(input("Enter server port number: "))
16 except:
17     print("Invalid input. Using default port 12000.")
18     serverPort = 12000
19
20 serverSocket = socket(AF_INET, SOCK_STREAM)
21 serverSocket.bind(("", serverPort))
22 serverSocket.listen(5)
23 print("The multithreaded server is ready to receive")
24
25 # Threading implementation #

```

Listing 3: Client socket implementation in C

```

1  int main(void) {
2      printf("---- AUTOMATED MULTITHREADED CLIENT ----\n\n");
3      srand(time(NULL));
4      printf("Enter server IP address (e.g., 127.0.0.1): ");
5      if (fgets(server_ip, sizeof(server_ip), stdin) != NULL) { //
6          Cleaning Cywing output.
7          server_ip[strcspn(server_ip, "\r\n")] = '\0';
8      }
9      pthread_t threads[NUM_CLIENTS];
10     for (int i = 0; i < NUM_CLIENTS; i++) {
11         thread_data_t *data = malloc(sizeof(thread_data_t));
12         data->client_id = i + 1;
13         if (pthread_create(&threads[i], NULL, client_thread, data) != 0)
14             {
15                 fprintf(stderr, "Failed to create thread %d\n", i + 1);
16             }
17     }
18     for (int i = 0; i < NUM_CLIENTS; i++) {
19         pthread_join(threads[i], NULL);
20     }
21     printf("All clients finished.\n");
22     return 0;
23 }

```

Listing 4: Server socket implementation in C

```

1  int main(void) {
2      printf("---- MULTITHREADED SERVER ----\n\n");
3      struct sockaddr_in sa;
4      int socket_fd;

```

```

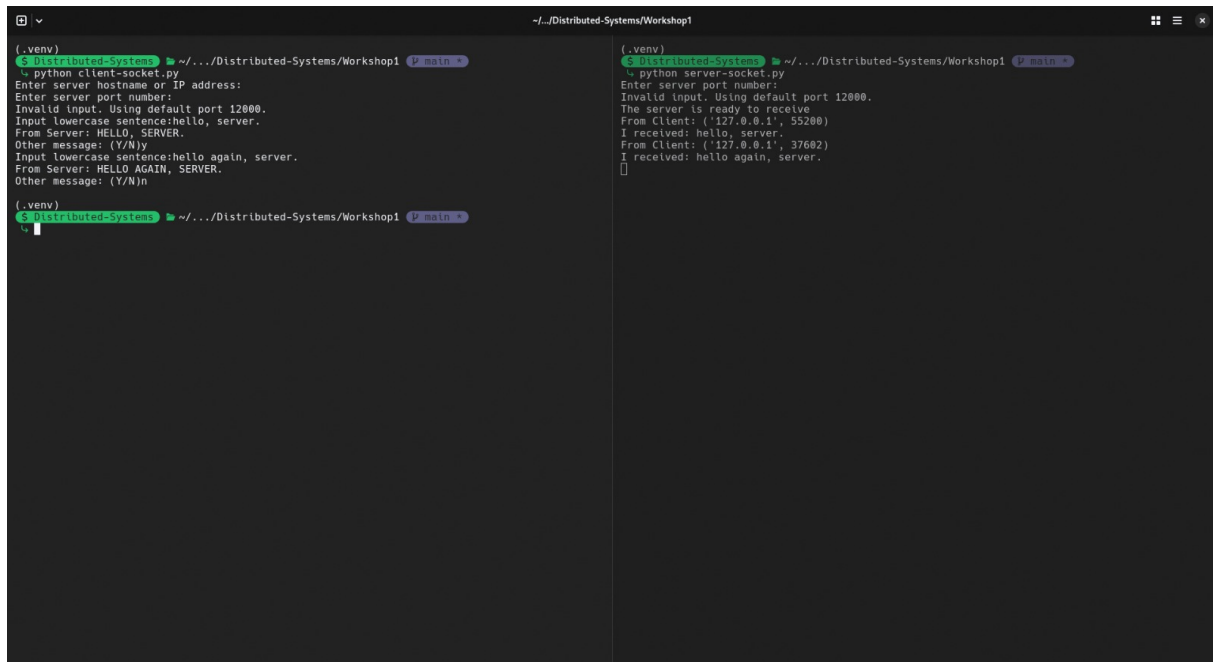
5  int status;
6  memset(&sa, 0, sizeof sa);
7  sa.sin_family = AF_INET;
8  sa.sin_addr.s_addr = htonl(INADDR_ANY); // Accept incoming
    connections from any IP address.
9  sa.sin_port = htons(PORT);
10 socket_fd = socket(sa.sin_family, SOCK_STREAM, 0);
11 if (socket_fd == -1) {
12     fprintf(stderr, "socket_fd_error: %s\n", strerror(errno));
13     return (1);
14 }
15 int opt = 1; // Setup so the same port can be used immediately.
16 setsockopt(socket_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
17 status = bind(socket_fd, (struct sockaddr *)&sa, sizeof sa);
18 if (status != 0) {
19     fprintf(stderr, "bind_error: %s\n", strerror(errno));
20     return (2);
21 }
22 printf("Listening on port %d\n", PORT);
23 status = listen(socket_fd, BACKLOG);
24 if (status != 0) {
25     fprintf(stderr, "listen_error: %s\n", strerror(errno));
26     return (3);
27 }
28 while (1) {
29     struct sockaddr_storage client_addr;
30     socklen_t addr_size = sizeof client_addr;
31     int *client_fd = malloc(sizeof(int)); // Assign memory for the
        descriptor.
32     *client_fd = accept(socket_fd, (struct sockaddr *)&client_addr,
        &addr_size);
33     if (*client_fd == -1) {
34         fprintf(stderr, "client_fd_error: %s\n", strerror(errno));
35         free(client_fd);
36         continue;
37     }
38     printf("Accepted new connection, spawning thread...\n");
39     pthread_t thread_id;
40     if (pthread_create(&thread_id, NULL, handle_client, client_fd)
        != 0) {
41         fprintf(stderr, "Failed to create thread\n");
42         free(client_fd);
43     }
44     pthread_detach(thread_id); // Free the resources used by the
        thread.
45 }
46 close(socket_fd);
47 return (0);
48 }

```

4 Experiments and Results

4.1 Activity 1: Localhost Testing

Both client and server were executed on the same machine using 127.0.0.1. The communication was successful.

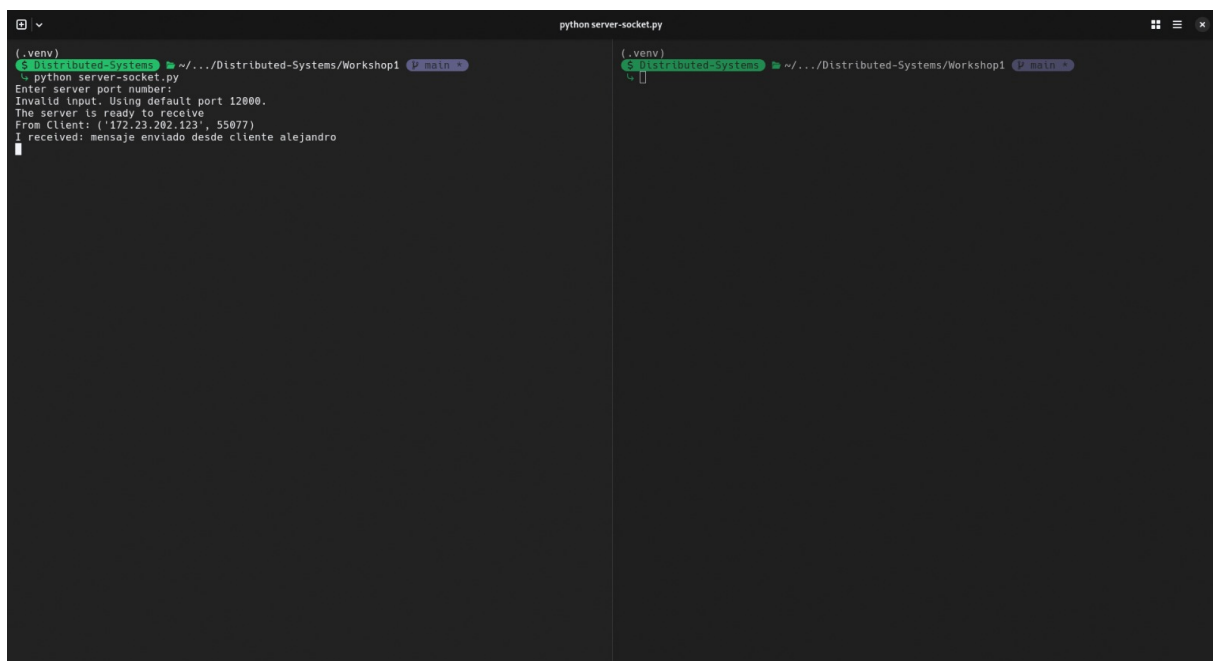


The image shows two terminal windows side-by-side. The left window is titled '~/.venv' and shows the execution of 'python client-socket.py'. It prompts for a server port number (12000), a server hostname or IP address (localhost), and a lowercase sentence ('hello, server.'). It then displays the server's response: 'HELLO, SERVER.' and 'HELLO AGAIN, SERVER.' The right window is also titled '~/.venv' and shows the execution of 'python server-socket.py'. It prompts for a server port number (12000) and displays the server's status: 'The server is ready to receive'. It then shows the client's connection details: 'From Client: ('127.0.0.1', 55200)' and the received message: 'I received: hello, server.'

Figure 1: Server and client running on localhost establishing basic communication.

4.2 Activity 2: Different Hosts

The server was executed on Linux and the client on Windows. After adjusting firewall rules, the client successfully connected using the server's LAN IP.



The image shows two terminal windows side-by-side. The left window is titled 'python server-socket.py' and shows the execution of 'python server-socket.py'. It prompts for a server port number (12000) and displays the server's status: 'The server is ready to receive'. It then shows the client's connection details: 'From Client: ('172.23.202.123', 55077)' and the received message: 'I received: mensaje enviado desde cliente alejandro'. The right window is titled '~/.venv' and shows the execution of 'python client-socket.py'. It prompts for a server port number (12000) and displays the server's response: 'HELLO, SERVER.' and 'HELLO AGAIN, SERVER.'

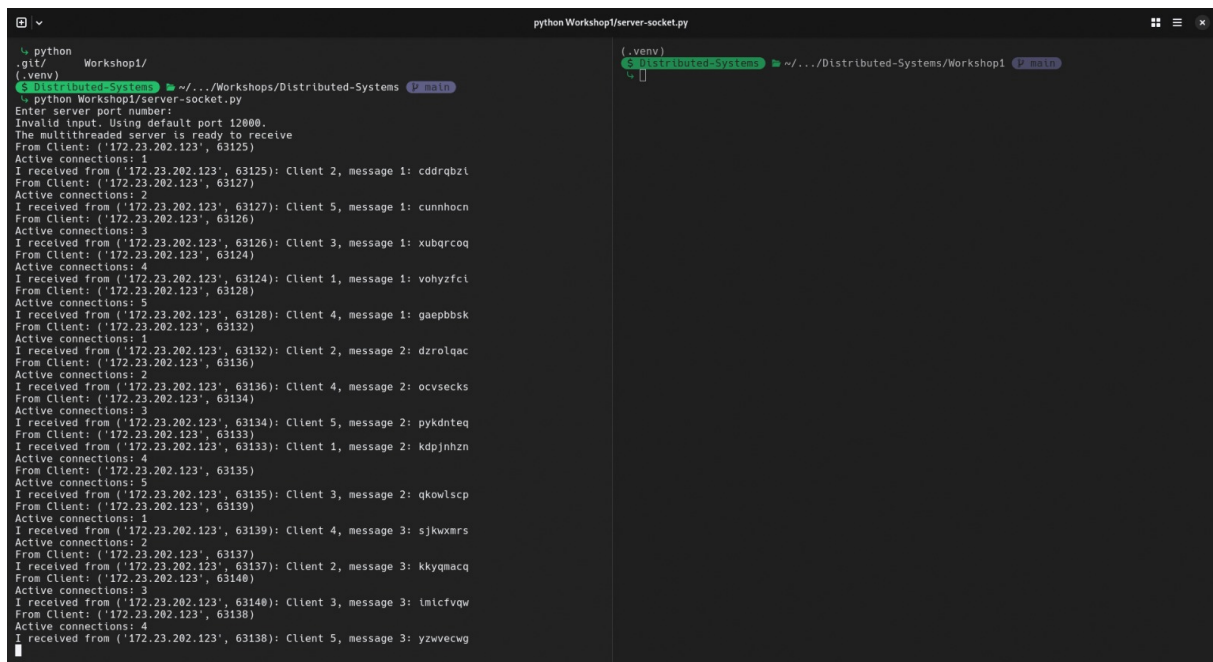
Figure 2: Python Server side running on Linux, receiving a message from a different host.

```
(.venv) PS C:\Users\alejo\SEMESTER_2_2026\DS> python .\client-socket.py
Enter server hostname or IP address: 172.23.197.112
Enter server port number:
Invalid input. Using default port 12000.
Input lowercase sentence:mensaje enviado desde cliente alejandro
From Server: MENSAJE ENVIADO DESDE CLIENTE ALEJANDRO
```

Figure 3: Python Client (windows) side successfully communicating with the remote server.

4.3 Activity 3, 4, 5: Multithreading and Automated Messages

Five concurrent client threads were launched from a single Python script. The server handled them simultaneously, returning the capitalized random strings.



```
python Workshop1/server-socket.py
(.venv)
python Workshop1/server-socket.py
Enter server port number:
Invalid input. Using default port 12000.
The multithreaded server is ready to receive
From Client: ('172.23.202.123', 63125)
Active connections: 1
I received from ('172.23.202.123', 63125): Client 2, message 1: cddrqbz1
From Client: ('172.23.202.123', 63127)
Active connections: 2
I received from ('172.23.202.123', 63127): Client 5, message 1: cunnhocn
From Client: ('172.23.202.123', 63126)
Active connections: 3
I received from ('172.23.202.123', 63126): Client 3, message 1: xubqrcq
From Client: ('172.23.202.123', 63124)
Active connections: 4
I received from ('172.23.202.123', 63124): Client 1, message 1: vohyzfci
From Client: ('172.23.202.123', 63128)
Active connections: 5
I received from ('172.23.202.123', 63128): Client 4, message 1: gaepbbsk
From Client: ('172.23.202.123', 63132)
Active connections: 1
I received from ('172.23.202.123', 63132): Client 2, message 2: dzrolqac
From Client: ('172.23.202.123', 63136)
Active connections: 2
I received from ('172.23.202.123', 63136): Client 4, message 2: ocvsecks
From Client: ('172.23.202.123', 63134)
Active connections: 3
I received from ('172.23.202.123', 63134): Client 5, message 2: pykdnteq
From Client: ('172.23.202.123', 63133)
I received from ('172.23.202.123', 63133): Client 1, message 2: kdpjnhzn
Active connections: 4
From Client: ('172.23.202.123', 63135)
Active connections: 5
I received from ('172.23.202.123', 63135): Client 3, message 2: qkowlscp
From Client: ('172.23.202.123', 63139)
Active connections: 1
I received from ('172.23.202.123', 63139): Client 4, message 3: sjkwxmrs
From Client: ('172.23.202.123', 63137)
I received from ('172.23.202.123', 63137): Client 2, message 3: kkyqmacq
From Client: ('172.23.202.123', 63140)
Active connections: 3
I received from ('172.23.202.123', 63140): Client 3, message 3: lmicfvqv
From Client: ('172.23.202.123', 63138)
Active connections: 4
I received from ('172.23.202.123', 63138): Client 5, message 3: yzwvecwg
```

Figure 4: Multithreaded Python server actively handling multiple connections and returning responses.

```

PS C:\Users\alejo\SEMESTER_2_2026\DS\Distributed-Systems\Workshop1> python client-socket.py
Enter the number of clients: 5
Enter server hostname or IP address:
2026-08-20 17:58:02,855 - INFO - No IP address specified. Using default address: 172.23.197.112
Enter server port number:
2026-08-20 17:58:05,547 - INFO - No port number specified. Using default port number: 12000
2026-08-20 17:58:05,548 - INFO - Preparing to connect to the server...
2026-08-20 17:58:05,763 - INFO - Client 1 successfully connected to the server.
2026-08-20 17:58:05,763 - INFO - Client 3 successfully connected to the server.
2026-08-20 17:58:05,763 - INFO - Client 2 successfully connected to the server.
2026-08-20 17:58:05,763 - INFO - Client 4 successfully connected to the server.
2026-08-20 17:58:05,763 - INFO - Client 5 successfully connected to the server.
[Client 2] Message from server: CLIENT 2, MESSAGE 1: CDDRQBZI
[Client 1] Message from server: CLIENT 1, MESSAGE 1: VOHYZFIC
[Client 5] Message from server: CLIENT 5, MESSAGE 1: CUNNHOCN
[Client 3] Message from server: CLIENT 3, MESSAGE 1: XUBQRCOQ
[Client 4] Message from server: CLIENT 4, MESSAGE 1: GAEPBBSK
2026-08-20 17:58:08,828 - INFO - Client 2 successfully connected to the server.
2026-08-20 17:58:08,830 - INFO - Client 5 successfully connected to the server.
2026-08-20 17:58:08,830 - INFO - Client 1 successfully connected to the server.
2026-08-20 17:58:08,830 - INFO - Client 3 successfully connected to the server.
2026-08-20 17:58:08,830 - INFO - Client 4 successfully connected to the server.
[Client 2] Message from server: CLIENT 2, MESSAGE 2: DZROLQAC
[Client 5] Message from server: CLIENT 5, MESSAGE 2: PYKDNTAQ
[Client 4] Message from server: CLIENT 4, MESSAGE 2: OCVSECKS
[Client 3] Message from server: CLIENT 3, MESSAGE 2: QKOWLSQP
[Client 1] Message from server: CLIENT 1, MESSAGE 2: KDPJNHZN
2026-08-20 17:58:12,279 - INFO - Client 4 successfully connected to the server.
2026-08-20 17:58:12,279 - INFO - Client 5 successfully connected to the server.
2026-08-20 17:58:12,279 - INFO - Client 3 successfully connected to the server.
2026-08-20 17:58:12,279 - INFO - Client 2 successfully connected to the server.
[Client 4] Message from server: CLIENT 4, MESSAGE 3: SJKWXMRS
[Client 2] Message from server: CLIENT 2, MESSAGE 3: KKYQMACQ
[Client 3] Message from server: CLIENT 3, MESSAGE 3: IMICFVQW
[Client 5] Message from server: CLIENT 5, MESSAGE 3: YZWECWQ
2026-08-20 17:58:15,389 - INFO - All clients have finished.

```

Figure 5: Automated Python client generating threads and receiving responses simultaneously.

4.4 Activity 6 & 7: C Implementation and Cross-Language Testing

The C versions were successfully compiled and executed. Furthermore, we tested a Python client sending requests to the C server, proving that socket communication is language-agnostic at the OS level.

```

Message received from client 9: "Client 6, message 1: ngwcyqus"
Message received from client 6: "Client 4, message 1: yirbmaxt"
Message received from client 4: "Client 2, message 1: nlbycsae"
[Thread] Client socket 9 closed connection.
[Thread] Client socket 5 closed connection.
[Thread] Closed client socket 5
[Thread] Client socket 6 closed connection.
[Thread] Client socket 4 closed connection.
[Thread] Closed client socket 6
[Thread] Closed client socket 4
[Thread] Client socket 7 closed connection.
[Thread] Client socket 8 closed connection.
[Thread] Closed client socket 7
[Thread] Closed client socket 8
[Thread] Closed client socket 9
Accepted new connection, spawning thread...
Accepted new connection, spawning thread...
[Thread] Handling client socket 4
Message received from client 4: "Client 2, message 2: mmkmhckb"
Accepted new connection, spawning thread...
Accepted new connection, spawning thread...
[Thread] Handling client socket 6
[Thread] Handling client socket 10
[Thread] Handling client socket 5
Message received from client 5: "Client 1, message 2: jsvkljqa"
Message received from client 6: "Client 5, message 2: fscevvyj"
Message received from client 10: "Client 3, message 2: gdrwicut"
[Thread] Client socket 5 closed connection.
[Thread] Client socket 4 closed connection.
[Thread] Closed client socket 5
[Thread] Closed client socket 4
[Thread] Client socket 6 closed connection.
[Thread] Closed client socket 6
[Thread] Client socket 10 closed connection.
[Thread] Closed client socket 10
Accepted new connection, spawning thread...
[Thread] Handling client socket 7
Message received from client 7: "Client 2, message 3: gudxptyo"
Accepted new connection, spawning thread...
Accepted new connection, spawning thread...
[Thread] Handling client socket 4
Message received from client 4: "Client 1, message 3: uzcamwpt"
[Thread] Handling client socket 5
Message received from client 5: "Client 3, message 3: wpvbxkab"
[Thread] Client socket 5 closed connection.
[Thread] Client socket 4 closed connection.
[Thread] Closed client socket 5
[Thread] Closed client socket 4
[Thread] Client socket 7 closed connection.
[Thread] Closed client socket 7

```

Figure 6: Multithreaded C Server handling connections and printing messages received from the remote Python client.

```

PS C:\Users\alejo\SEMESTER_2_2026\DS\Distributed-Systems\Workshop1> python client-socket.py
Enter the number of clients: 6
Enter server hostname or IP address: 172.23.197.112
Enter server port number:
2026-08-20 21:13:43,296 - INFO - No port number specified. Using default port number: 12000
2026-08-20 21:13:43,297 - INFO - Preparing to connect to the server...
2026-08-20 21:13:45,510 - INFO - Client 1 successfully connected to the server.
2026-08-20 21:13:45,511 - INFO - Client 3 successfully connected to the server.
2026-08-20 21:13:45,510 - INFO - Client 5 successfully connected to the server.
2026-08-20 21:13:45,510 - INFO - Client 4 successfully connected to the server.
2026-08-20 21:13:45,510 - INFO - Client 6 successfully connected to the server.
2026-08-20 21:13:45,510 - INFO - Client 2 successfully connected to the server.
[Client 1] Message from server: CLIENT 1, MESSAGE 1: RJWOSGOW
[Client 3] Message from server: CLIENT 3, MESSAGE 1: TYXTMKYS
[Client 4] Message from server: CLIENT 4, MESSAGE 1: YIRBMAXT
[Client 6] Message from server: CLIENT 6, MESSAGE 1: NGWCYQUS
[Client 5] Message from server: CLIENT 5, MESSAGE 1: AHNSMKDE
[Client 2] Message from server: CLIENT 2, MESSAGE 1: NLBYCSAE
2026-08-20 21:13:46,663 - INFO - Client 2 successfully connected to the server.
2026-08-20 21:13:46,663 - INFO - Client 1 successfully connected to the server.
2026-08-20 21:13:46,663 - INFO - Client 5 successfully connected to the server.
2026-08-20 21:13:46,664 - INFO - Client 3 successfully connected to the server.
[Client 1] Message from server: CLIENT 1, MESSAGE 2: JSVKLJQA
[Client 2] Message from server: CLIENT 2, MESSAGE 2: MMKMHCCKB
[Client 5] Message from server: CLIENT 5, MESSAGE 2: FSCEVVYJ
[Client 3] Message from server: CLIENT 3, MESSAGE 2: GDRWICUT
2026-08-20 21:13:47,680 - INFO - Client 1 successfully connected to the server.
2026-08-20 21:13:47,680 - INFO - Client 2 successfully connected to the server.
2026-08-20 21:13:47,680 - INFO - Client 3 successfully connected to the server.
[Client 2] Message from server: CLIENT 2, MESSAGE 3: GUDXPTYO
[Client 1] Message from server: CLIENT 1, MESSAGE 3: UZCAMWPT
[Client 3] Message from server: CLIENT 3, MESSAGE 3: WPVBXKAB
2026-08-20 21:13:48,790 - INFO - All clients have finished.

```

Figure 7: Python client connecting to the remote C server and receiving the processed uppercase messages.

5 Results Summary

Client Lang	Server Lang	Execution Hosts	Clients	Success
Python	Python	Localhost (Windows)	1	Yes
Python	Python	Windows (Server) - Linux (Client)	1	Yes
Python	Python	Windows (Server) - Linux (Client)	5 (Concurrent)	Yes
C	C	Localhost (Cygwin)	3 (Concurrent)	Yes
Python	C	Linux (Server) - Windows (Client)	5 (Concurrent)	Yes

Table 1: Summary of Workshop Activities

6 Problems and Solutions

During the execution of the workshop, several networking and synchronization issues were encountered:

- **[WinError 10061] Connection Refused / Timeout:** Initially, the Linux client could not reach the Windows server. This was solved by changing the Windows network profile from "Public" to "Private" and explicitly allowing inbound traffic on TCP port 12000 in the Windows Defender Firewall.

7 Discussion

The implementation of multithreading proved essential for distributed architectures. Regarding programming languages, Python provided a higher-level abstraction that made string manipulation and threading straightforward. In contrast, C required manual memory management, strict type enforcement, and explicit pointer handling for thread arguments, but resulted in a highly optimized executable.

8 Conclusions

This workshop successfully demonstrated the fundamentals of socket programming. We learned how to establish TCP connections, overcome local firewall and routing restrictions, and handle concurrent connections using multithreading in both POSIX and Windows environments. The interoperability test between Python and C reinforced the concept that network protocols operate independently of the application layer's programming language.

9 References

- Python Software Foundation. *socket — Low-level networking interface*. Retrieved from <https://docs.python.org/3/library/socket.html>

10 Appendix - Source Code

The complete, and organized source code for all activities (Python and C versions) can be found in our public GitHub repository:

GitHub Repository: <https://github.com/demivn15/Distributed-Systems>